Course Outline: Object Oriented Programming

Prerequisites: Introduction to Programming

Course Code:7121

Course Learning Outcomes

Upon successful completion of this course, students will be able to:

1. **Design, write, and debug** OOP programs that utilize fundamental programming constructs such as variables, control structures, loops, and functions.
2. **Apply problem-solving skills** to decompose complex problems into smaller, manageable sub-problems that can be implemented as modular functions.
3. **Explain and implement** fundamental data structures like arrays and structs, and demonstrate a core understanding of memory management through the use of pointers and dynamic memory allocation.
4. **Describe the principles of Object-Oriented Programming (OOP)** and create simple classes and objects with member variables and methods to model real-world entities.

Assessment Breakdown

- **Sitting CAT 1:** 20%
- **CAT2 :** 20%
- **Final Exam :** 60%

Week Java Programming Course Outline

Week 1: Introduction to Java & Development Environment

Topics & Subtopics:

- History and features of Java
- Installing JDK and setting up IDE (IntelliJ/Eclipse)
- Writing your first Java program

Learning Outcomes:

1. Explain the purpose and evolution of Java.
2. Identify key features that make Java platform-independent.
3. Set up a Java development environment.
4. Write and run a basic Java program.

Activities:

- Install JDK and IDE, and configure environment variables.
- Create and run a “Hello, World!” program.

Week 2: Variables, Data Types & Operators

Topics & Subtopics:

- Primitive and reference data types
- Type casting
- Arithmetic, relational, logical operators

Learning Outcomes:

1. Declare and initialize variables using different data types.
2. Apply type casting between compatible types.
3. Use operators to perform calculations and comparisons.
4. Debug common errors related to data types.

Activities:

- Create a calculator using arithmetic operators.
- Write a program that demonstrates type conversion.

Week 3: Control Flow Statements

Topics & Subtopics:

- Conditional statements (if, else, switch)
- Looping constructs (for, while, do-while)
- Nested loops and conditions

Learning Outcomes:

1. Implement decision-making using conditional statements.
2. Use loops to repeat actions based on conditions.
3. Combine loops and conditions for complex logic.
4. Analyze flow control in real-world scenarios.

Activities:

- Build a number guessing game.

- Create a program to print multiplication tables.

Week 4: Methods and Arrays

Topics & Subtopics:

- Defining and calling methods
- Method parameters and return types
- Arrays: declaration, initialization, traversal

Learning Outcomes:

1. Define reusable methods with parameters and return values.
2. Apply method overloading.
3. Create and manipulate arrays.
4. Use methods to process array data.

Activities:

- Write a method to find the average of an array.
- Create a program that sorts an array using a method.

Week 5: Object-Oriented Programming – Classes & Objects

Topics & Subtopics:

- Defining classes and creating objects
- Constructors and this keyword
- Instance vs. static members

Learning Outcomes:

1. Create classes and instantiate objects.
2. Use constructors to initialize object state.
3. Differentiate between instance and static variables.
4. Apply encapsulation principles.

Activities:

- Build a Student class with attributes and methods.
- Create multiple objects and display their data.

Week 6: Inheritance and Polymorphism

Topics & Subtopics:

- extends keyword and superclass-subclass relationships
- Method overriding
- super keyword
- Polymorphism and dynamic method dispatch

Learning Outcomes:

1. Implement inheritance to reuse code.
2. Override methods to customize behavior.
3. Use polymorphism for flexible code design.
4. Apply super to access superclass members.

Activities:

- Create a class hierarchy for Animal, Dog, and Cat.
- Demonstrate polymorphism using overridden methods.

Week 7: Abstraction and Interfaces

Topics & Subtopics:

- Abstract classes and methods
- Interfaces and multiple inheritance
- Functional interfaces (Java 8+)

Learning Outcomes:

1. Define abstract classes and implement them.
2. Create and use interfaces.
3. Understand the role of abstraction in design.
4. Apply interfaces in real-world scenarios.

Activities:

- Design a payment system using interfaces.
- Implement an abstract class for a vehicle hierarchy.

Week 8: Exception Handling

Topics & Subtopics:

- try, catch, finally blocks
- throw and throws keywords
- Custom exceptions

Learning Outcomes:

1. Handle runtime errors using exception handling.
2. Use try-catch-finally effectively.
3. Create and throw custom exceptions.
4. Debug programs using exception messages.

Activities:

- Build a program that validates user input and handles exceptions.
- Create a custom exception for invalid login attempts.

Week 9: Collections Framework

Topics & Subtopics:

- Lists, Sets, Maps
- Iterators and enhanced for loop
- Sorting and searching collections

Learning Outcomes:

1. Use Java collections to store and manipulate data.
2. Iterate over collections using different techniques.
3. Sort and search data using built-in methods.
4. Compare ArrayList, HashSet, and HashMap.

Activities:

- Create a contact manager using HashMap.
- Build a program to sort student grades using ArrayList.

Week 10: File I/O and Final Project

Topics & Subtopics:

- Reading and writing files
- BufferedReader and FileWriter

- Final project development

Learning Outcomes:

1. Read and write text files using Java I/O classes.
2. Handle file exceptions gracefully.
3. Apply all learned concepts in a final project.
4. Present and explain project design and implementation.

Activities:

- Write a program to log user activity to a file.
- Final project: Build a console-based library management system.

Books

1. Parsons, D. (2020). *Foundational Java: Key Elements and Practical Programming*. Springer. https://doi.org/10.1007/978-3-030-54518-5
2. Eck, D. J. (2021). *Introduction to Programming Using Java* (8th ed.). Open Textbook Library. https://open.umn.edu/opentextbooks/textbooks/419
3. Moore, A. (2023). *Java Programming Essentials: Learn Java 17 and Beyond*. CodersGuild. https://codersguild.net/books/java

Additional Resources

- *Introduction to Java programming by Daniel Liang.*
- *Java-How to program by Deitel Deitel.*